University of Antwerp
Faculty of Science

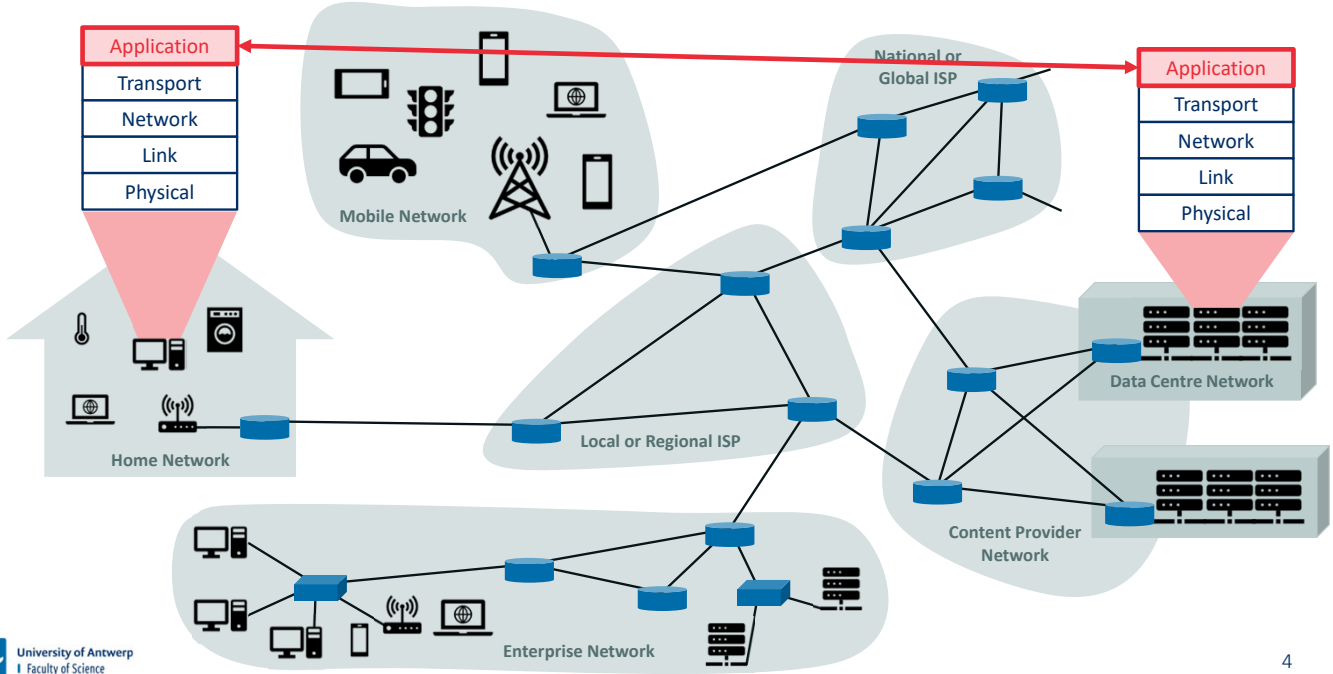
Computer Networks (2025-2026)
Part 2: Application Layer

Jeroen Famaey
jeroen.famaey@uantwerpen.be

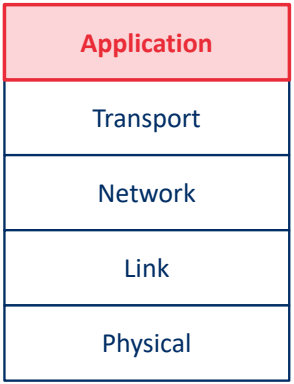
Table of contents

Application layer principles

Network applications run on end systems



Application layer

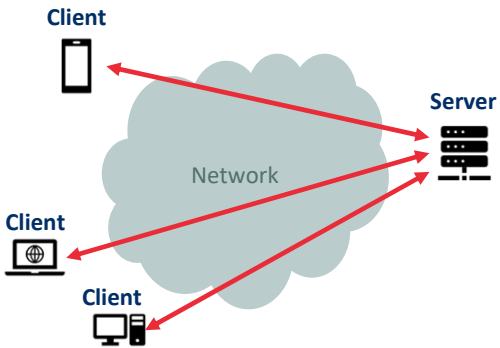


Application Layer

- Distributed over multiple end-systems
- Consists of the applications and their associated application-layer protocols
- Examples
  - HyperText Transfer Protocol (HTTP)
  - Simple Message Transfer Protocol (SMTP)
  - File Transfer Protocol (FTP)
  - Domain Name System (DNS)
  - ...
- Packet of information is referred to as a **message** at this layer

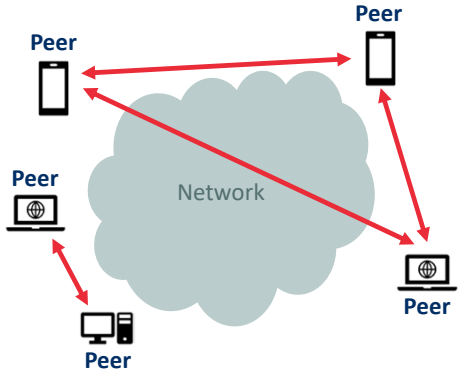
Network application architecture

Client-server architecture



- **Server** is always on (with a well-known IP address)
- **Clients** do not directly communicate with each other
- Data centres offer powerful virtual servers
- Examples: Web, FTP, Telnet, e-mail

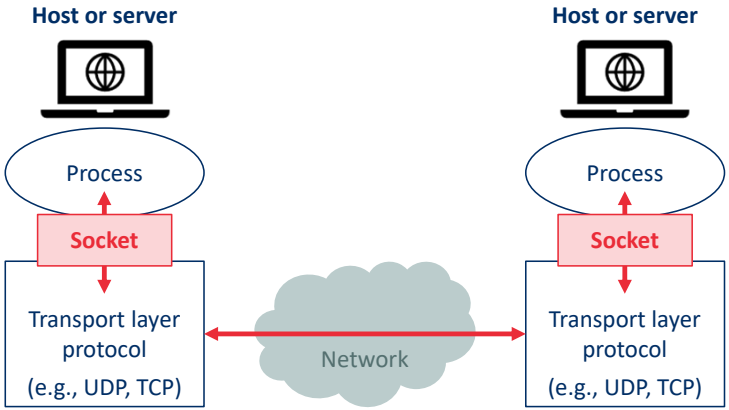
Peer-to-peer architecture



- Minimal or no reliance on always-on servers
- Direct communication between **peers**
- Self-scaling and cost-effective
- Examples: BitTorrent, Bitcoin network, Skype (hybrid)

Communication between processes

- **Client**: Process that initiates the communication session between a pair of processes
- **Server**: Process that waits to be contacted to begin the session
- **Socket**: The application programming interface (API) between the application and transport layer
- Host is identified by an **IP address** and process by a **port number**

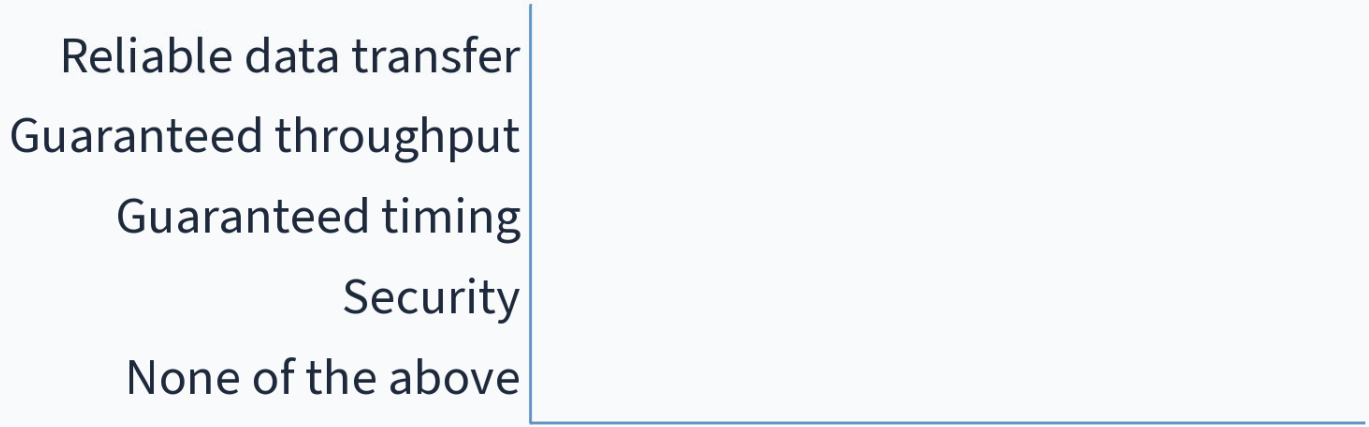


Transport layer services

Service types a transport layer **could** offer to applications:

- **Reliable data transfer**: Ensures that data sent by one process will arrive at the receiving process
- **Throughput**: Guarantees minimum available throughput at a specified rate (in bits/second)
- **Timing**: Guarantees a maximum end-to-end delay for transmitted data (in seconds)
- **Security**: Ensures security and privacy (e.g., confidentiality, integrity, authentication) of data

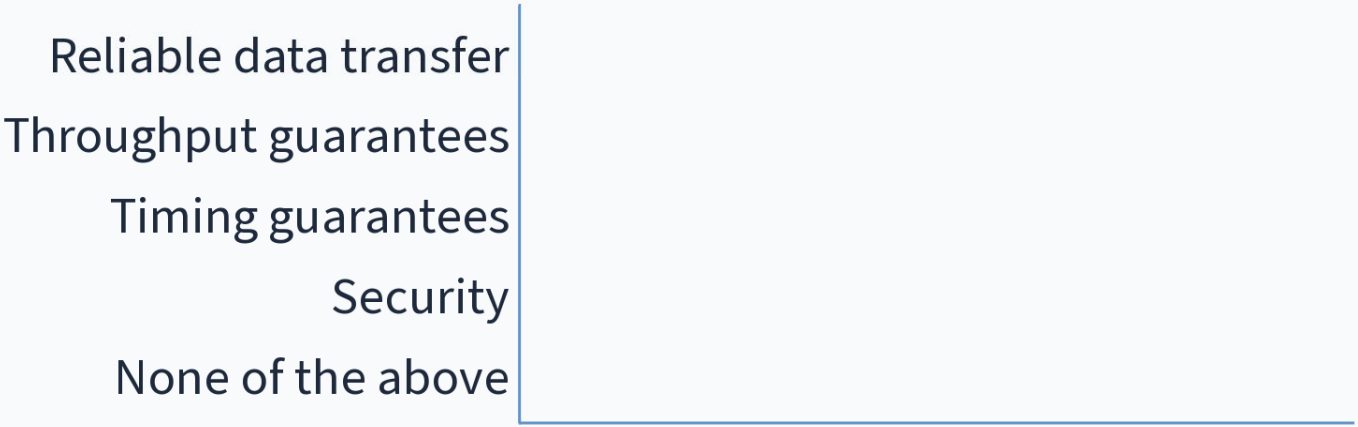
Which transport layer services are required for video conferencing applications?



Transport layer requirements of example applications

Application	Data Loss	Throughput	Time-Sensitive
File transfer or download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic	No
Internet telephony / video conferencing	Loss-tolerant	Audio: few kbps – 1 Mbps Video: 10 kbps – 5 Mbps	Yes: 100s of msec
Streaming stored audio / video	Loss-tolerant	Audio: few kbps – 1 Mbps Video: 10 kbps – 20 Mbps	Yes: a few seconds
Interactive games	Loss-tolerant	Few kbps – 5 Mbps	Yes: 10s – 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

Which transport layer services are provided by the Internet's TCP protocol?



Transport services provided by the Internet

Transport Layer Protocol (TCP)

- **Connection-oriented service**: Before transmitting data, a connection is set up via a handshake
- **Reliable data transfer service**: All data is guaranteed to be delivered in order and without error or loss
- **Congestion control**: Throttles sender to avoid network congestion and fairly share available bandwidth
- **Security:** Confidentiality, integrity and host authentication are offered through the TLS protocol

User Datagram Protocol (UDP)

- **Connectionless**: No handshake needed to start communication
- **Unreliable data transfer service**: Data may be lost or arrive out of order
- **No congestion control**: UDP processes can send as much data as they want/can

The Internet's transport layer **does not offer** throughput or timing guarantees!

Application-layer protocols

Defines type, syntax, and semantics of messages, as well as rules of message exchange

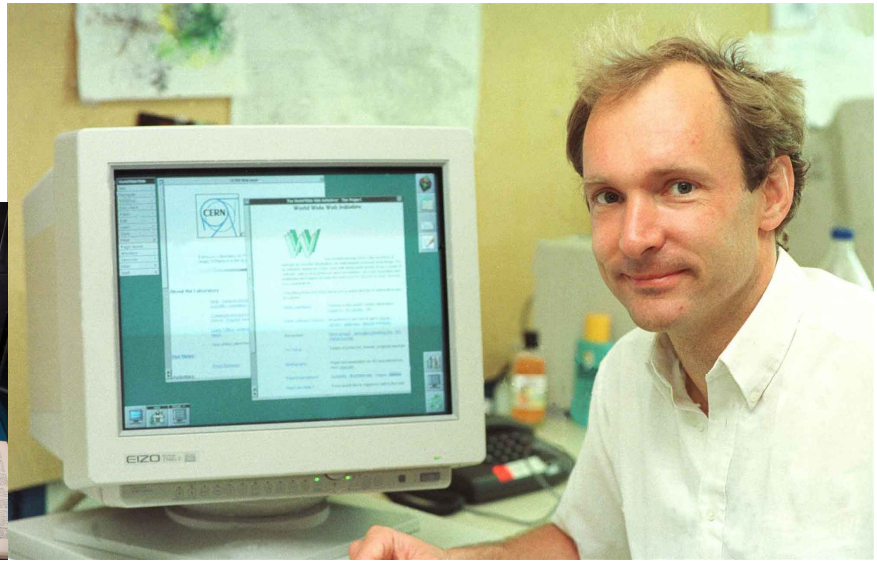
Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP 1.1 [RFC 7230], HTTP 2 [RFC 7540]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP, DASH	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], ...	UDP or TCP

May use TCP to circumvent firewalls that block UDP

The web and HTTP

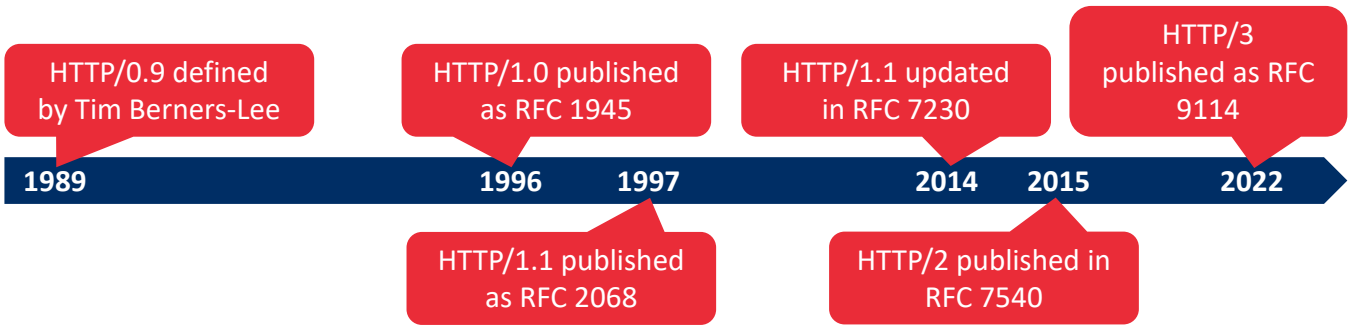
World Wide Web (WWW)

- Invented by Tim Berners-Lee between 1989 and 1991 at CERN
- Offers homogeneous information sharing using heterogeneous hardware and software

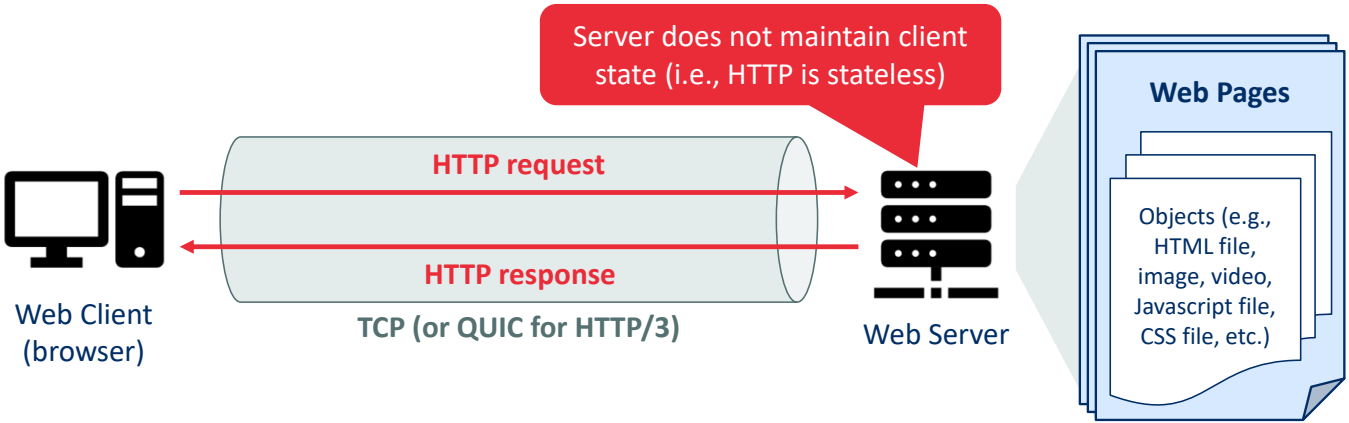


History of HyperText Transfer Protocol (HTTP)

The HTTP protocol defines the rules of communication between web clients (browsers) and servers

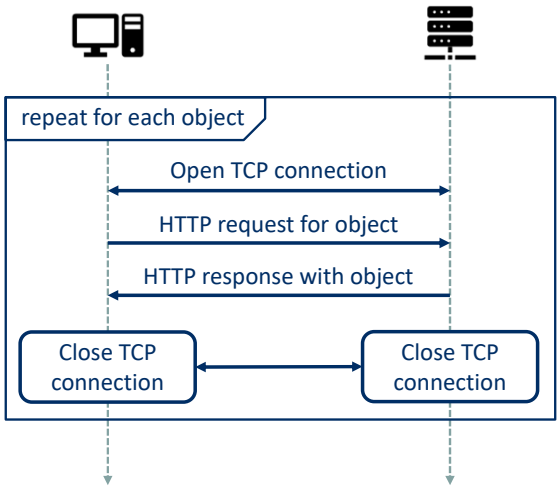


HTTP at a high level

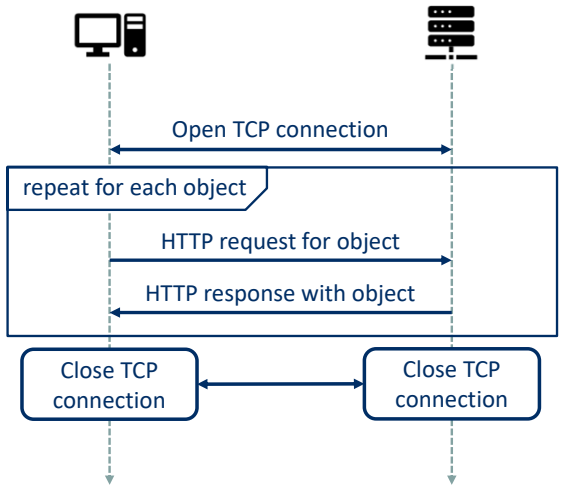


Persistent versus non-persistent connections

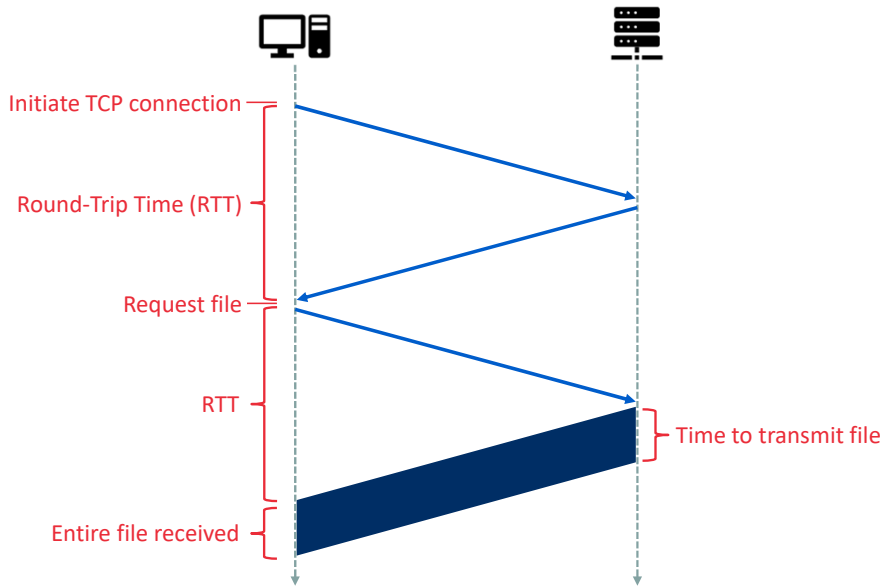
Non-persistent connection (HTTP/1.0)



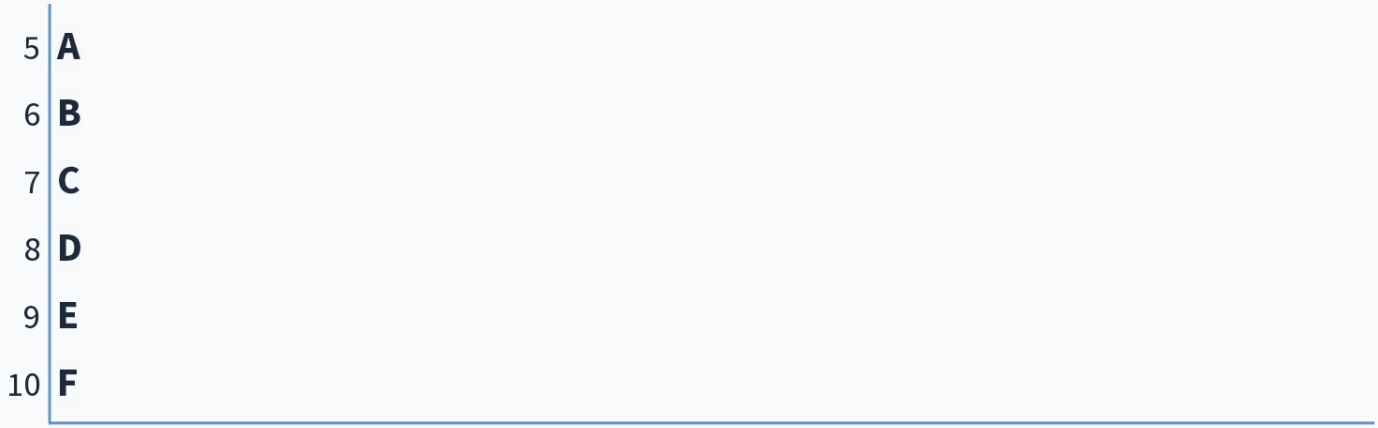
Persistent connection (default in HTTP/1.1)



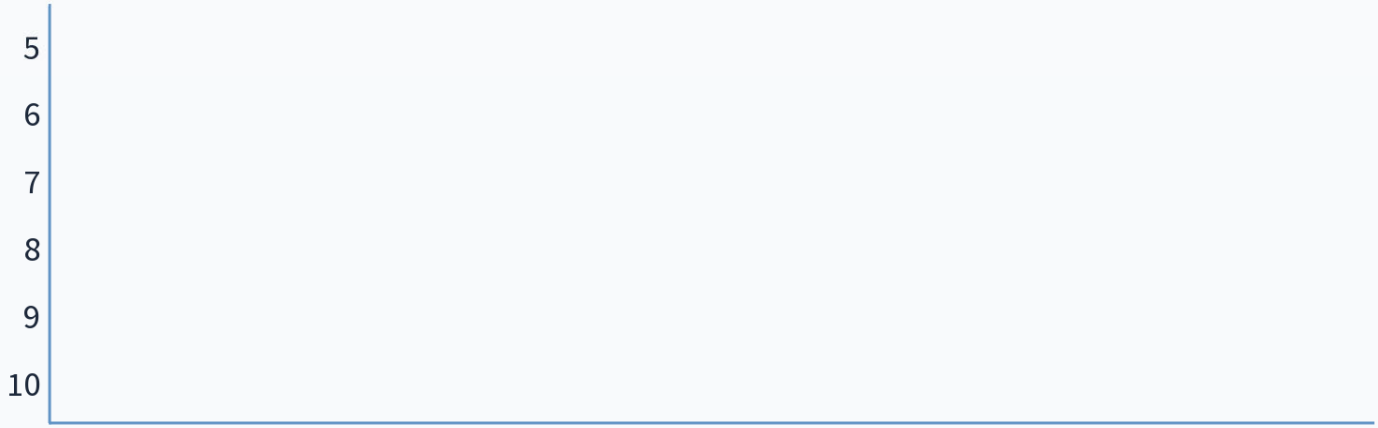
Setting up TCP connections affects delay



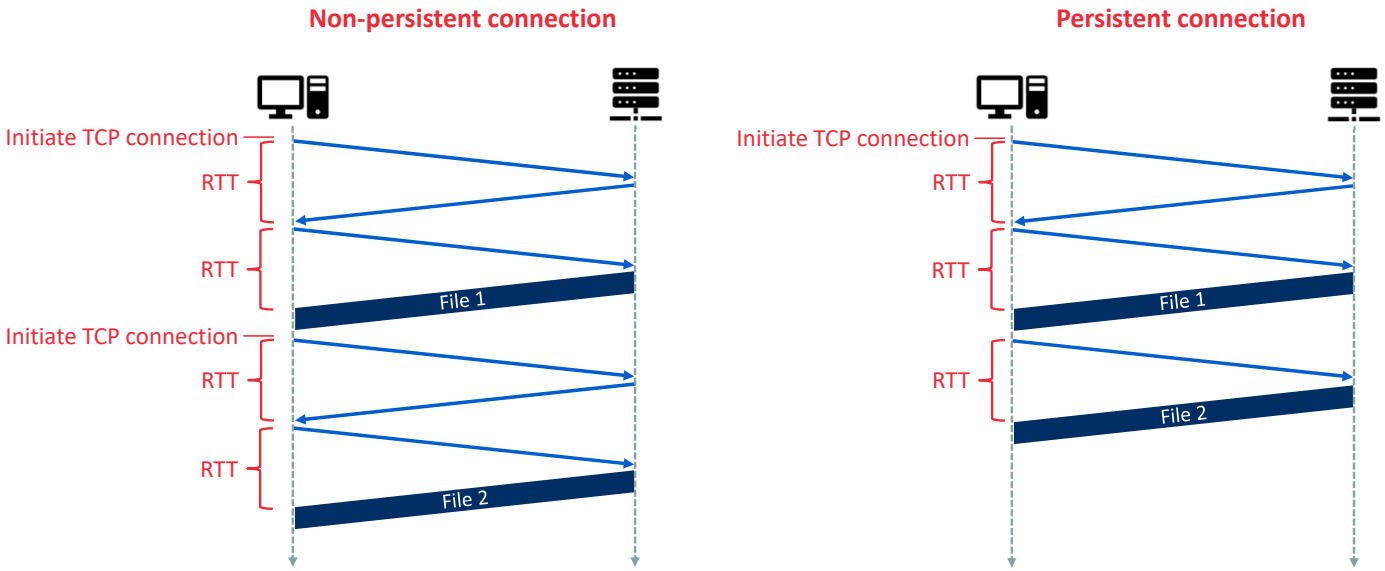
How many RTTs does it take to download 5 files using non-persistent HTTP?



How many RTTs does it take to download 5 files using persistent HTTP?

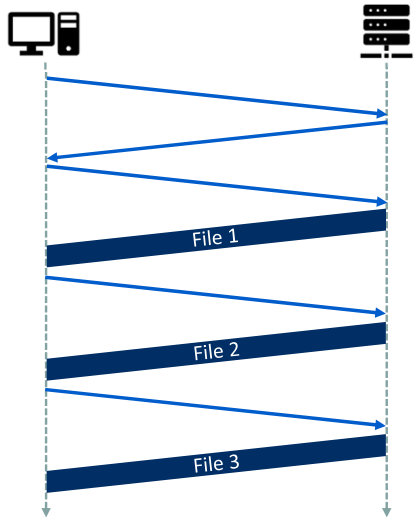


Delay when requesting multiple files

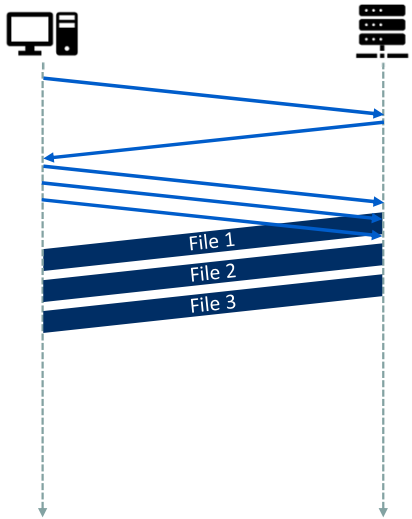


HTTP/1.1 supports pipelining to further reduce RTT

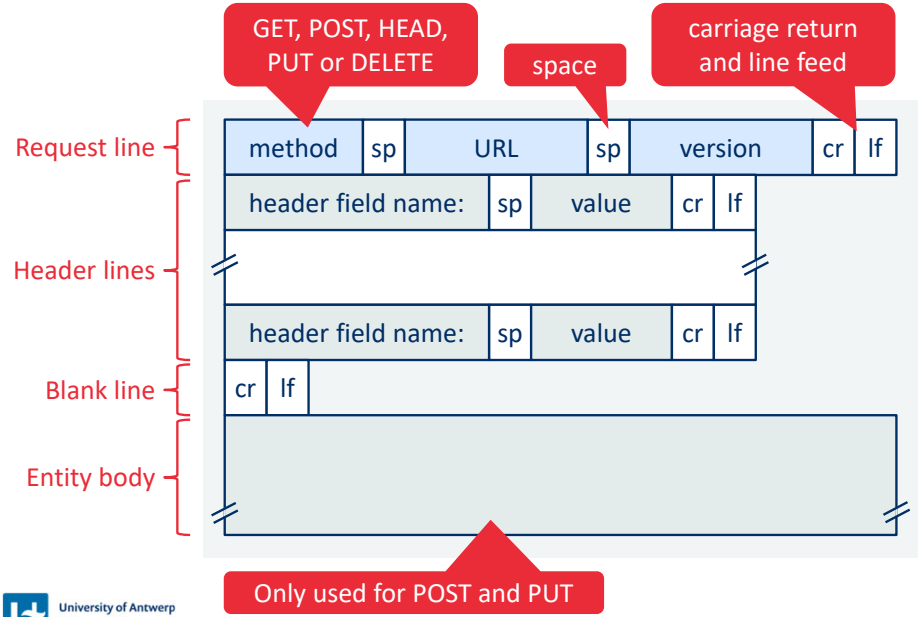
Persistent connection (no pipelining)



Persistent connection (pipelining)



HTTP request message format



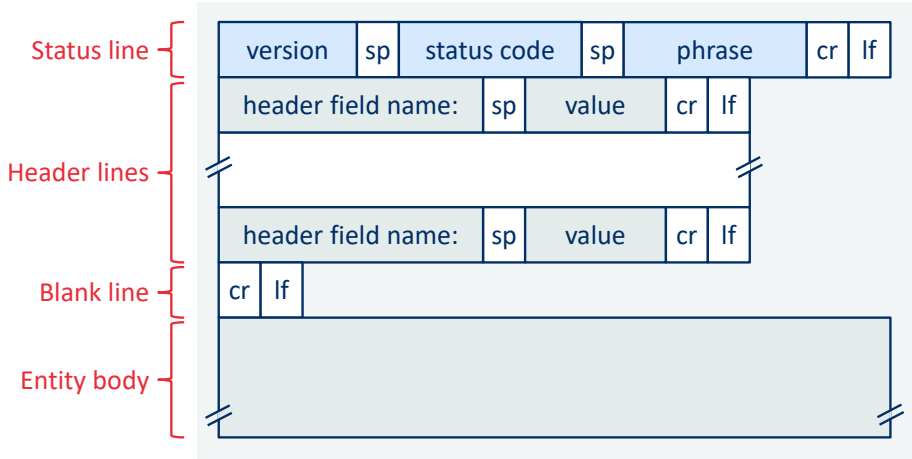
Example:

```
GET /somedir/page.html HTTP/1.1

Host: www.uantwerpen.be
Connection: close
User-agent: Mozilla/5.0
Accept-language: nl
```

HTTP request methods

- **GET**
  - Retrieves the information identified by the request URL
  - May be used to pass form parameters via the URL (e.g., http://www.uantwerpen.be?param1=val1&param2=val2)
- **HEAD**
  - Identical to GET, except that the server will not return the message body in the response
- **POST**
  - Used to pass parameters to the server via the request body (e.g., values filled out in a form)
  - The returned information will depend on the request URL and the passed parameter values
- **PUT**
  - Allows the user to upload an object (provided in the request body) to a specific path (specified by the URL) on the web server
- **DELETE**
  - Allows the user, or an application, to delete an object (specified by the URL) on a web server
- Other methods: OPTIONS, TRACE, CONNECT

HTTP response message



Example:

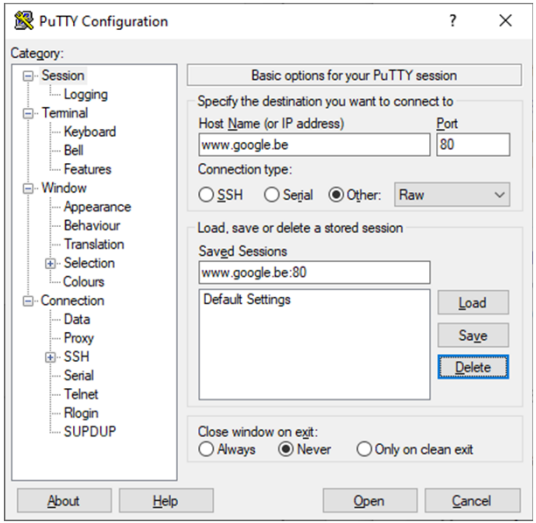
```
HTTP/1.1 200 OK

Connection: close
Date: 18 Aug 2015 15:44:04
Server: Apache/2.2.3 (CentOS)
Last-Modified: 18 Aug 2015 15:11:03
Content-Length: 6821
Content-Type: text/html


data
data
data
...
```

HTTP response status codes

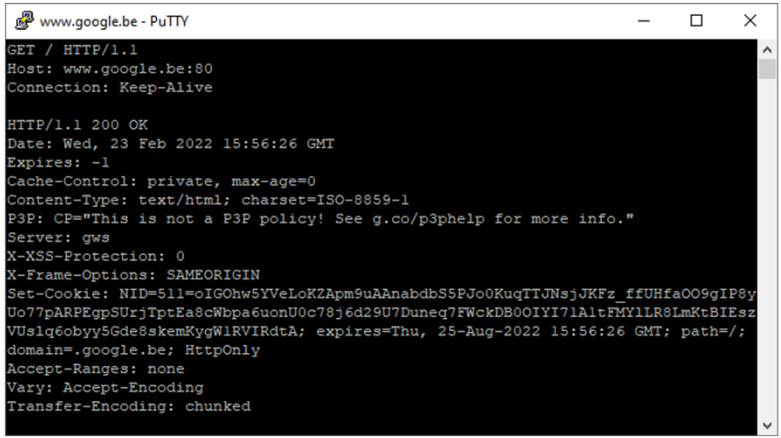
- **200 OK**
  - Request succeeded and information is returned in response
- **301 Moved Permanently**
  - Requested object has been permanently moved to a new URL (specified via the Location: header in the response)
- **400 Bad Request**
  - Generic error code indicating the server could not understand the request
- **404 Not Found**
  - The requested document does not exist on the server
- **505 HTTP Version Not Supported**
  - The requested HTTP protocol version is not supported by the server

Homework: hands-on testing

Open a raw TCP connection (e.g., using PuTTY):



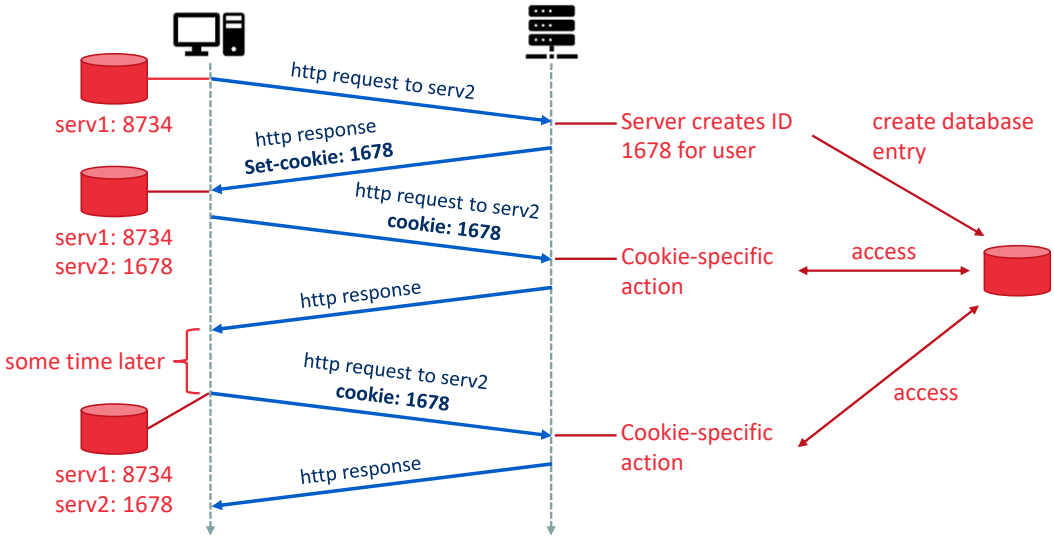
Request the main page ("/") of www.google.be at port 80:



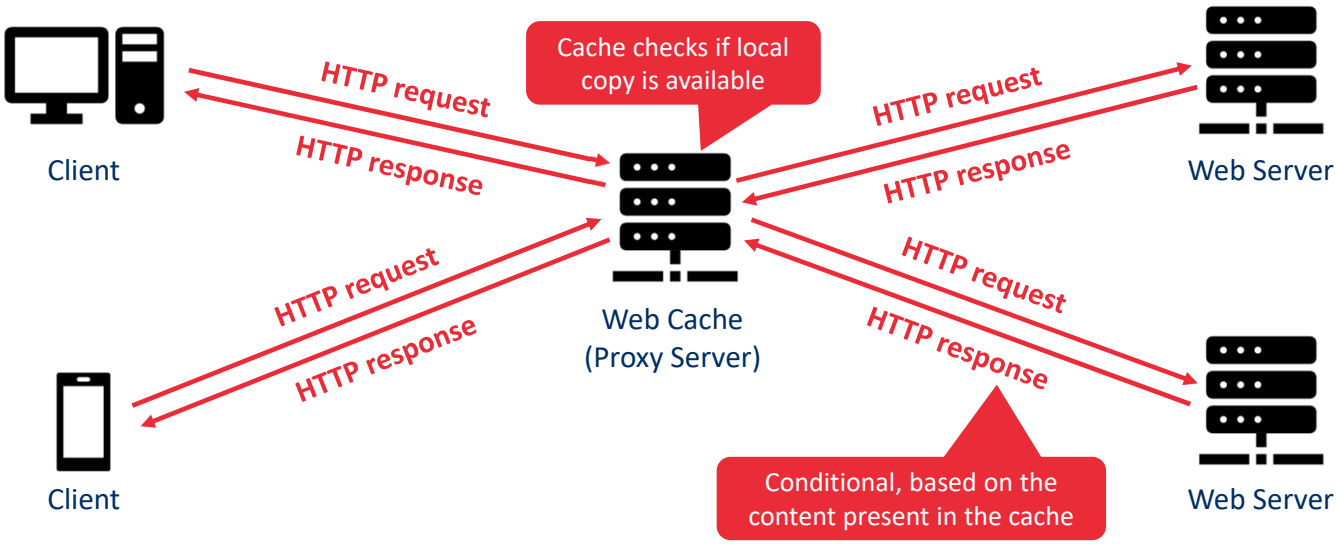
Homework: Try this out yourself, while capturing TCP traffic to or from port 80 with Wireshark

Tip: Use PuTTY (Windows), or netcat/nc (cross-platform) to open RAW TCP connection

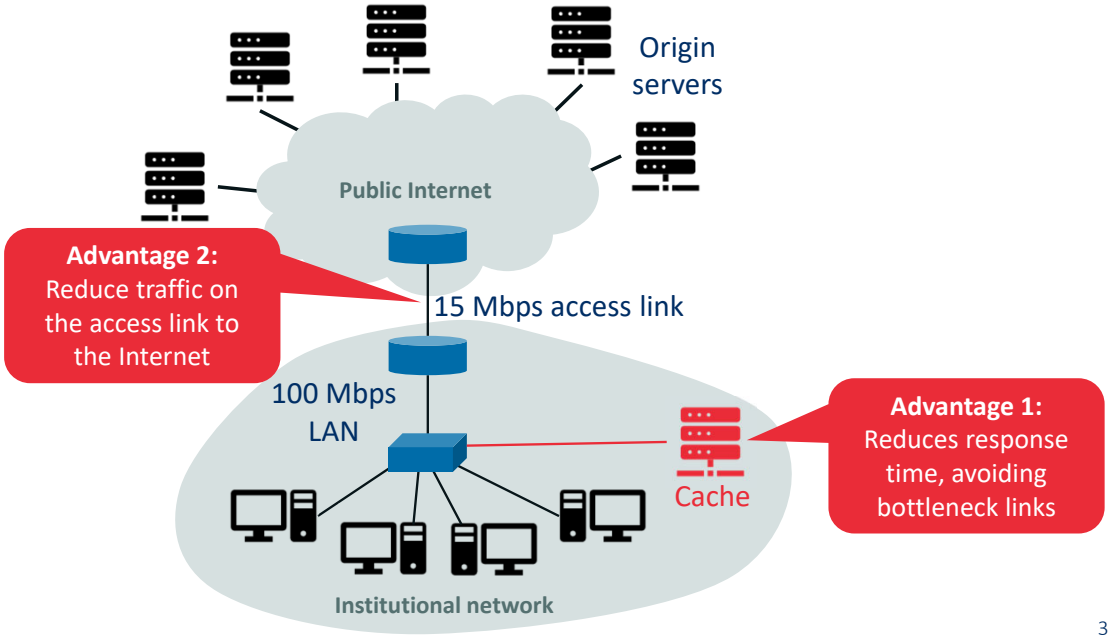
How to keep state with stateless HTTP protocol? Cookies!



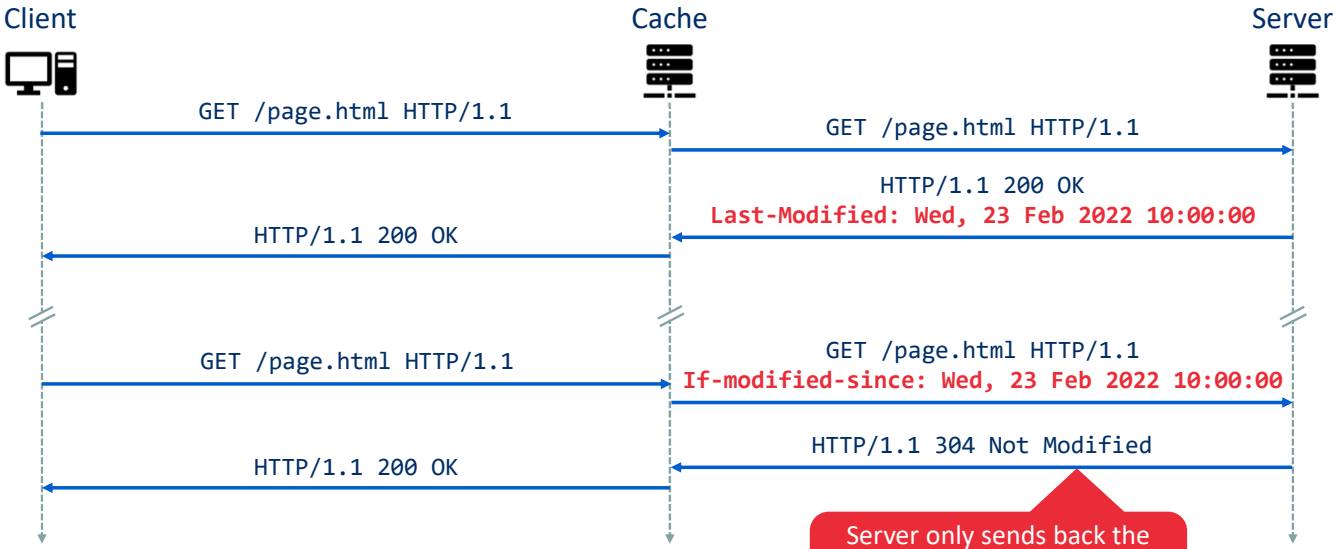
Web caches (also known as proxy servers)



Advantages of caching

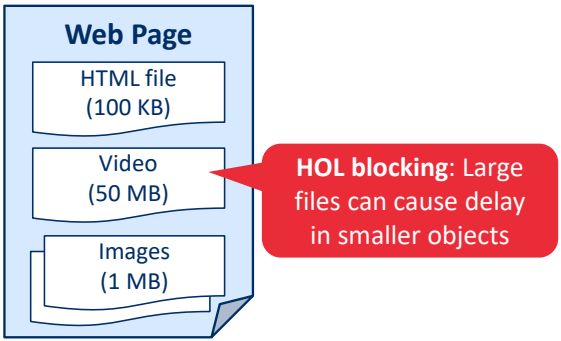


How does the cache verify if its copy is still up to date? Conditional GET!



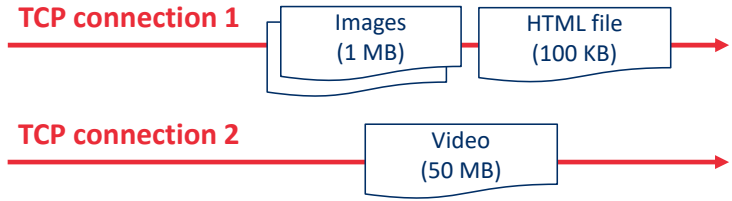
HTTP/2

- Does not change the HTTP methods, status codes, URLs or header fields over previous versions
- **Reduces perceived latency** through multiplexing, request prioritization, server push and compression
- Aims to solve **Head of Line (HOL) blocking** problem



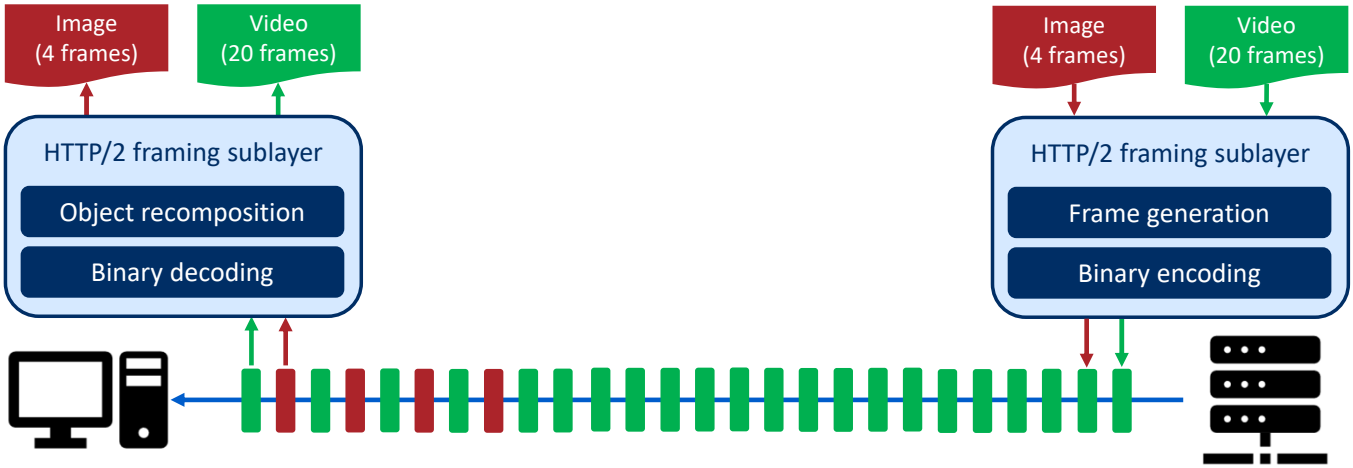
HTTP/1.1 solution:

- Open multiple simultaneous TCP connections
- Interferes with TCP congestion control and fairness mechanisms



HTTP/2 HOL blocking solution: Framing

- Each message (object) is broken into small frames
- Request and response frames are interleaved over a single TCP connection



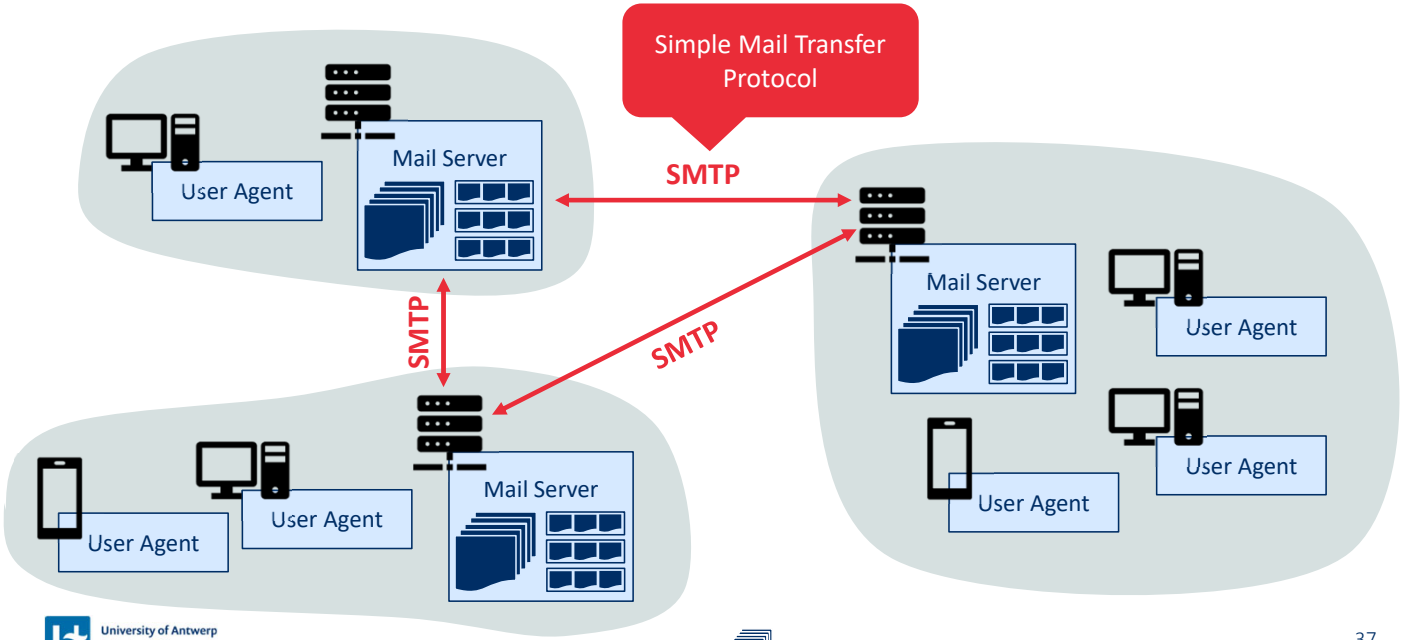
Other HTTP/2 features

- **Message Prioritization**
  - Allows web developers to customize the relative priority of requests
  - Each request is assigned a weight between 1 (lowest) and 256 (highest)
  - The server first sends the frames of responses with highest priority (interleaving those with equal priority)
  - Client can define dependencies between requests

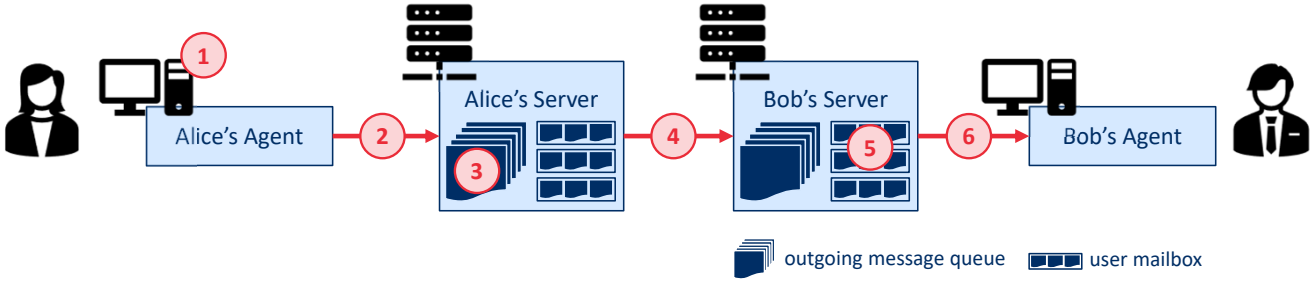
- **Server Push**
  - Allows the server to send multiple responses to a single request
  - Useful to pro-actively send additional content that is part of a requested HTML base page
  - Can also be used for pushing segmented video
  - Eliminates the extra latency due to waiting for multiple requests

Electronic mail (email)

High-level overview of Internet email

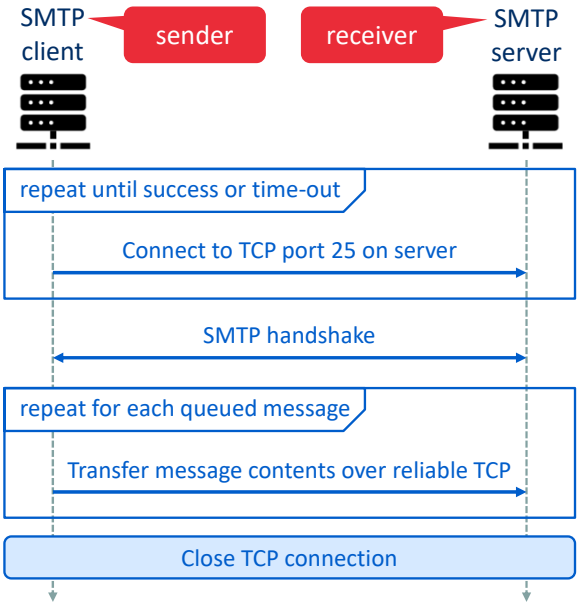


Simple Mail Transfer Protocol (SMTP) – RFC 5321



1. Alice composes an email to bob@emailprovider.com and instructs her users agent to send it
2. Alice's user agent sends the message to her mail server, where it is placed in the message queue
3. The SMTP client on Alice's mail server opens a TCP connection to the SMTP server on Bob's mail server
4. After the SMTP handshake, the SMTP client sends Alice's message via TCP
5. The SMTP server on Bob's mail server receives the message and places it into Bob's mailbox
6. Bob invokes his user agent to read the message at his convenience

Details of SMTP message transfer



Example

From client (C) crepes.fr to server (S) hamburger.edu:

Handshake

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
```

Message

```
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
```

Close connection

```
C: QUIT
S: 221 hamburger.edu closing connection
```

Internet Message Format – RFC 5322

- Represents the email contents (i.e., the DATA part in the sent SMTP message)
- Uses 7-bit ASCII encoding (like all SMTP messages)
- Consists of header fields (standardized in RFC 5322) and a message body
  - Header fields consist of a keyword, colon, and value
  - Header and body are separated by a blank line
- Example

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life.

MESSAGE DATA...
```

Multipurpose Internet Mail Extensions (MIME) – RFCs 2045 and 2046

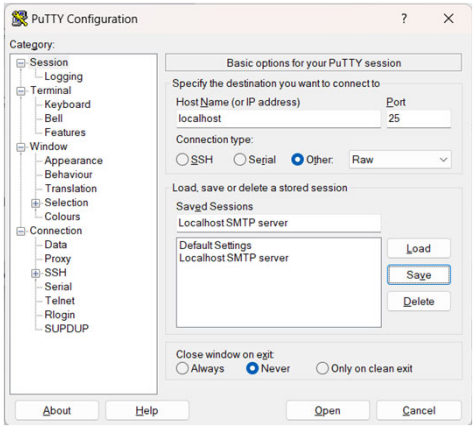
Type	Subtypes	Description
Text	Plain, enriched	Unformatted for formatted text
Multipart	Mixed, parallel, alternative, digest	Content of multiple types transmitted together
Message	Rfc822, partial, external-body	RFC822 compliant, large fragmented message, or pointer to external object
Image	Jpeg, gif	Image in JPEG or GIF format
Video	Mpeg	Video in MPEG format
Audio	Basic	Audio encoded at 8 kHz
Application	Postscript, octet-stream	Postscript file or general binary data

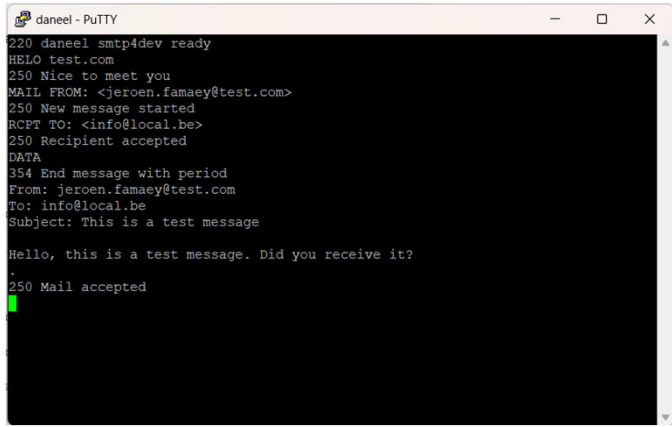
Example

```
From: Nathaniel Borenstein <nsb@bellcore.com>
To: Ned Freed <ned@innosoft.com>
Subject: Sample message
MIME-Version: 1.0
Content-type: multipart/mixed; boundary="simple boundary"

This is the preamble. It is to be ignored, though it is a handy
place for mail composers to include an explanatory note to non-
MIME conformant readers.
--simple boundary

This is implicitly typed plain ASCII text. It does NOT end with
a linebreak.
--simple boundary
Content-type: text/plain; charset=us-ascii

This is explicitly typed plain ASCII text. It DOES end with a
linebreak.

--simple boundary--
This is the epilogue. It is also to be ignored.
```

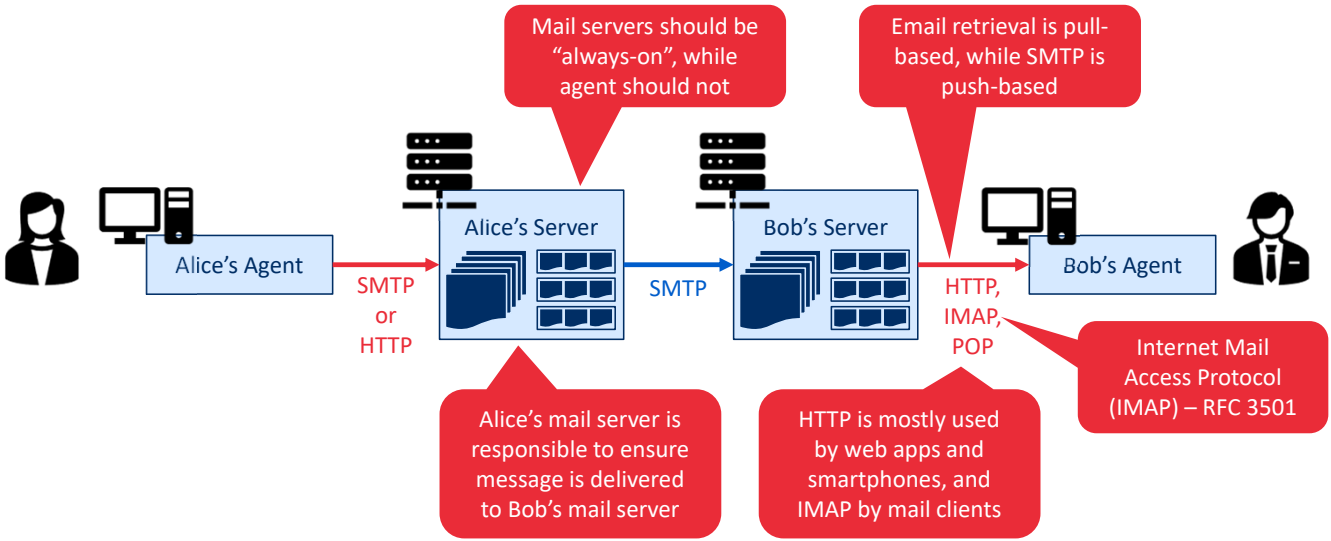
Homework: Testing out SMTP

- Easiest way to test SMTP is to install a local SMTP server (e.g., https://github.com/rnwood/smtp4dev, or MailHog)





Mail access protocols (i.e., how to access your mailbox?)



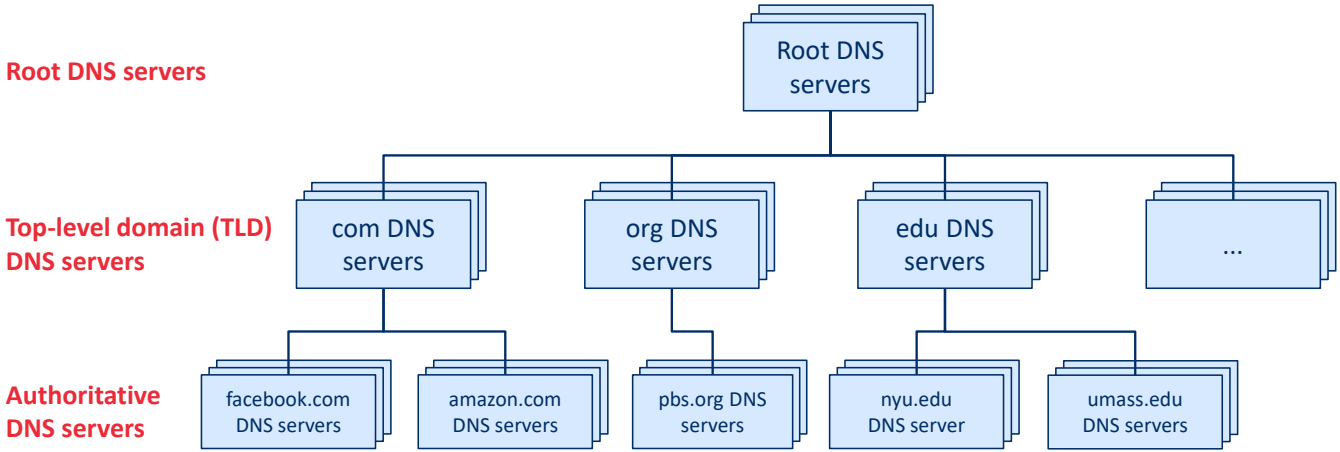
Domain Name System (DNS)

The Internet's Directory Service

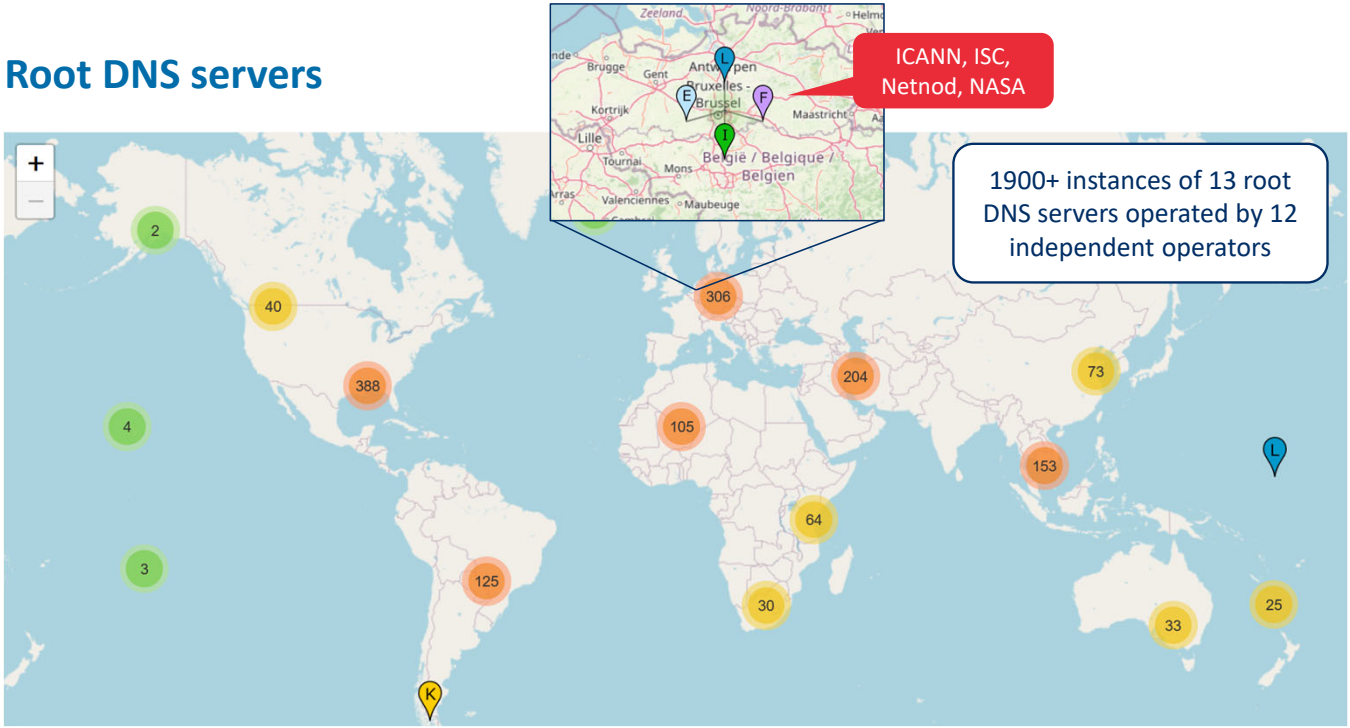
Domain name system (DNS) in a nutshell – RFCs 1034 and 1035

- Translates (human readable) hostnames to (router readable) IP addresses
- Consists of
  - A distributed hierarchy of DNS servers
  - An application-layer protocol that allows hosts to query the distributed database
- Makes use of UDP port 53
- Other services offered by DNS
  - **Host aliasing:** Defines a set of alternative (more mnemonic) hostnames in addition to the main **canonical hostname**
  - **Mail server aliasing:** Similar to host aliasing but for email addresses
  - **Load distribution:** Associating multiple IP addresses with 1 alias hostname to spread out traffic

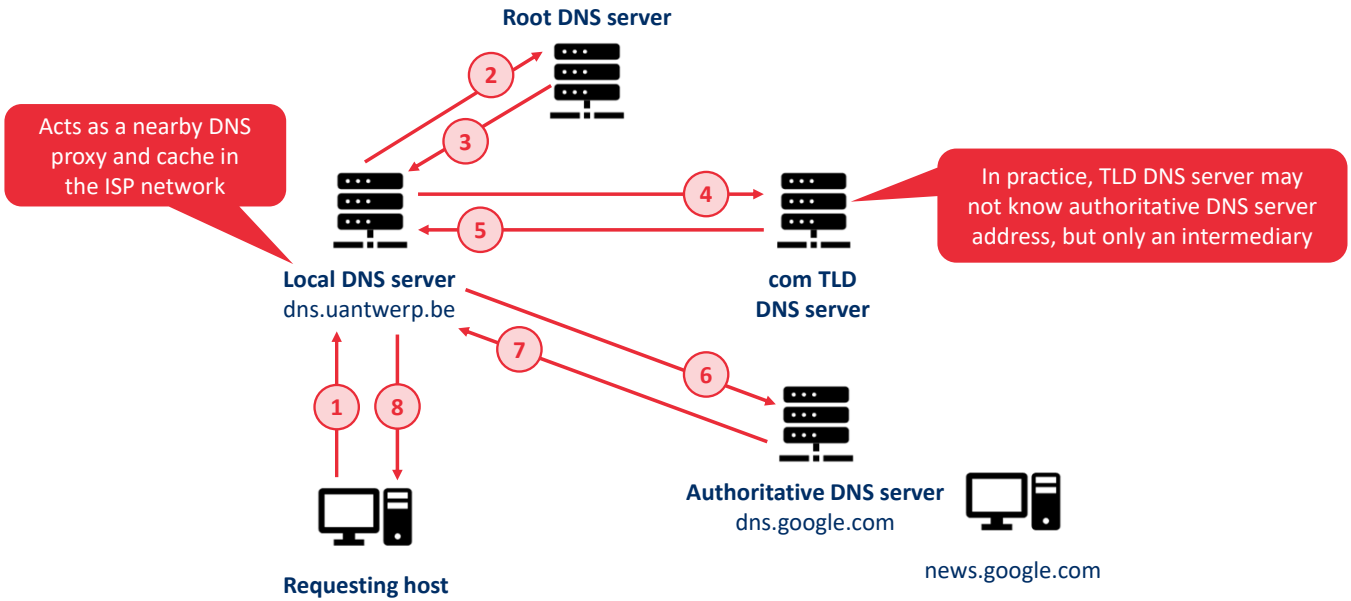
DNS is distributed in a hierarchical database



Root DNS servers

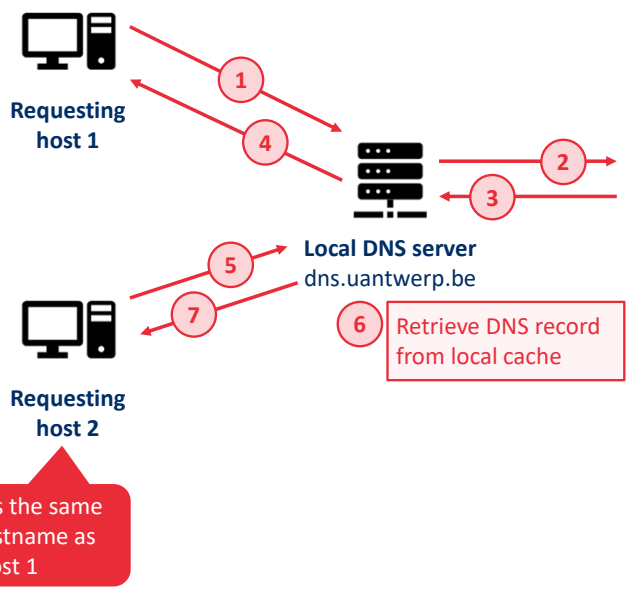


Interactions between DNS servers

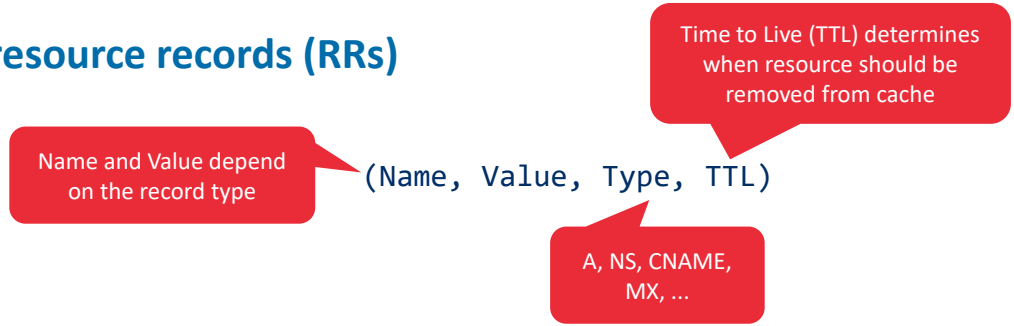


DNS caching

- Whenever a DNS server receives a DNS reply it can cache the mapping in local memory
- If another query arrives for the same hostname, the DNS server can immediately return the IP address
- DNS servers discard cached information after a period of time (usually 2 days)



DNS resource records (RRs)



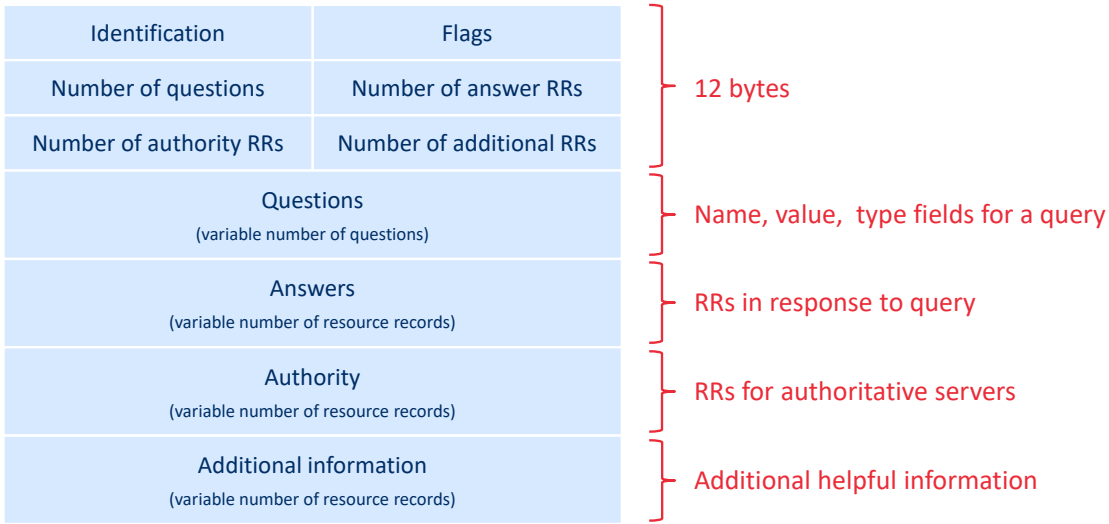
(most important) DNS record types:

- **A**: Provides a standard hostname-to-IP address mapping (Name is hostname and Value is IP address)
- **NS**: Used to route queries along the query chain (Name is domain and Value is authoritative DNS server)
- **CNAME**: Maps alias hostnames (Name) to canonical hostnames (Value)
- **MX**: Maps alias mail server names (Name) to canonical mail server names (Value)

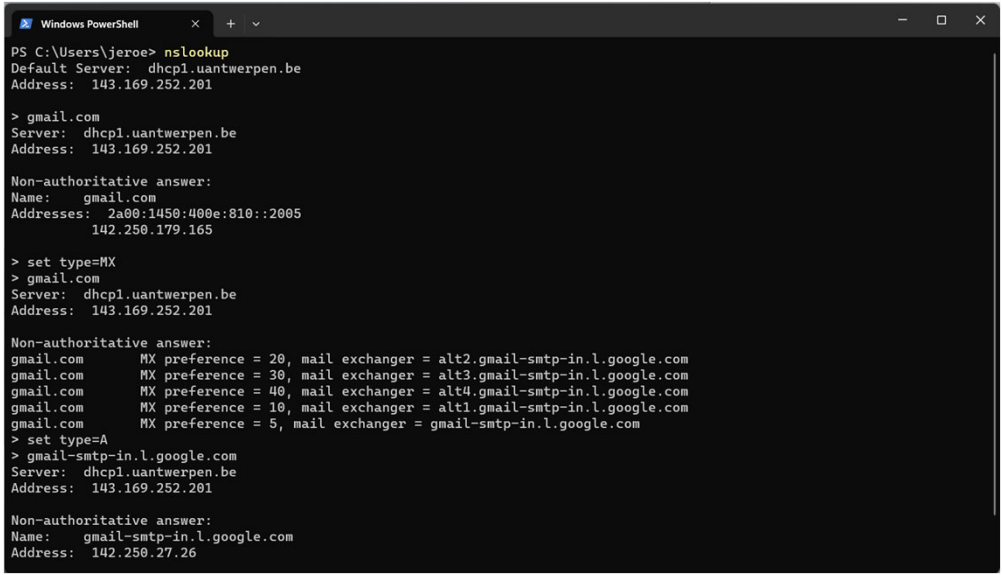
What is the correct DNS resource record format for an web server alias www.google.be for the canonical hostname google.be?

(google.be, www.google.be, CNAME, ttl)	**A**
(www.google.be, google.be, CNAME, ttl)	**B**
(google.be, www.google.be, MX, ttl)	**C**
(www.google.be, google.be, MX, ttl)	**D**

DNS messages (query and reply use the same format)



Homework: Performing DNS queries with *nslookup*



How are DNS records inserted into the DNS database?

Steps

1. Register your new domain name at a **registrar**
2. Provide primary and secondary authoritative DNS servers to registrar (or use theirs)
3. Registrar creates a Type NS and Type A record for each of the authoritative DNS servers
4. Enter Type A (for web server) and Type MX (for mail server) record into authoritative DNS servers

Example

dns1.mysite.com

(www.mysite.com, <web server IP>, A)

(mail.mysite.com, <mail server IP>, A)

(mysite.com, mail.mysite.com, MX)

dns2.mysite.com

(www.mysite.com, <web server IP>, A)

(mail.mysite.com, <mail server IP>, A)

(mysite.com, mail.mysite.com, MX)

com TLD DNS server

(mysite.com, dns1.mysite.com, NS)

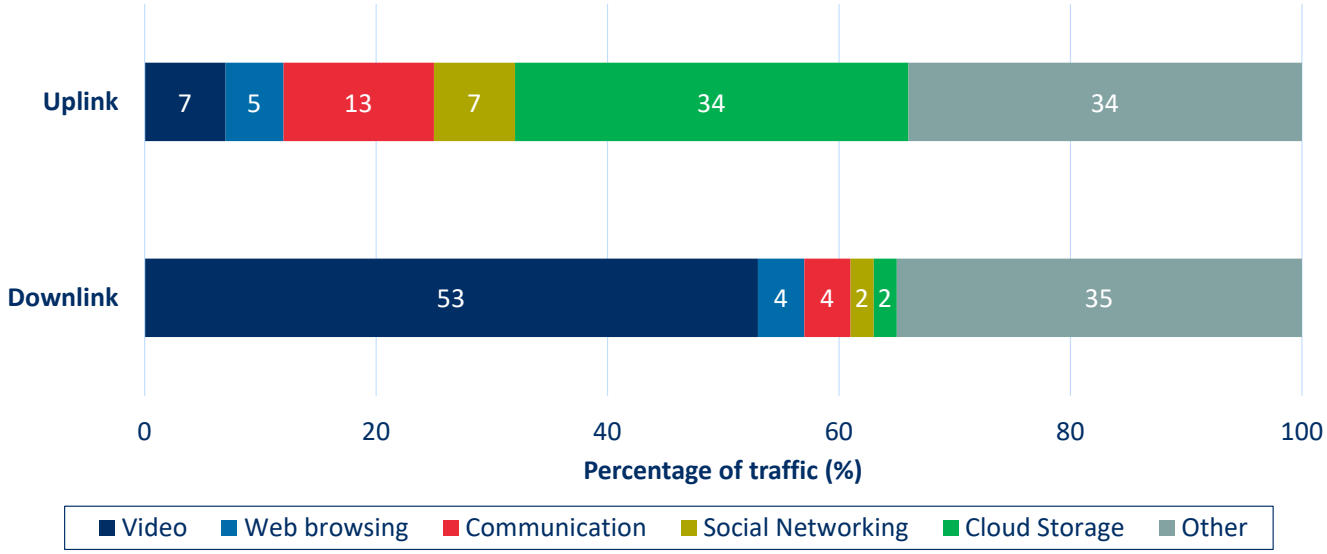
(mysite.com, dns2.mysite.com, NS)

(dns1.mysite.com, <DNS1 IP>, A)

(dns2.mysite.com, <DNS2 IP>, A)

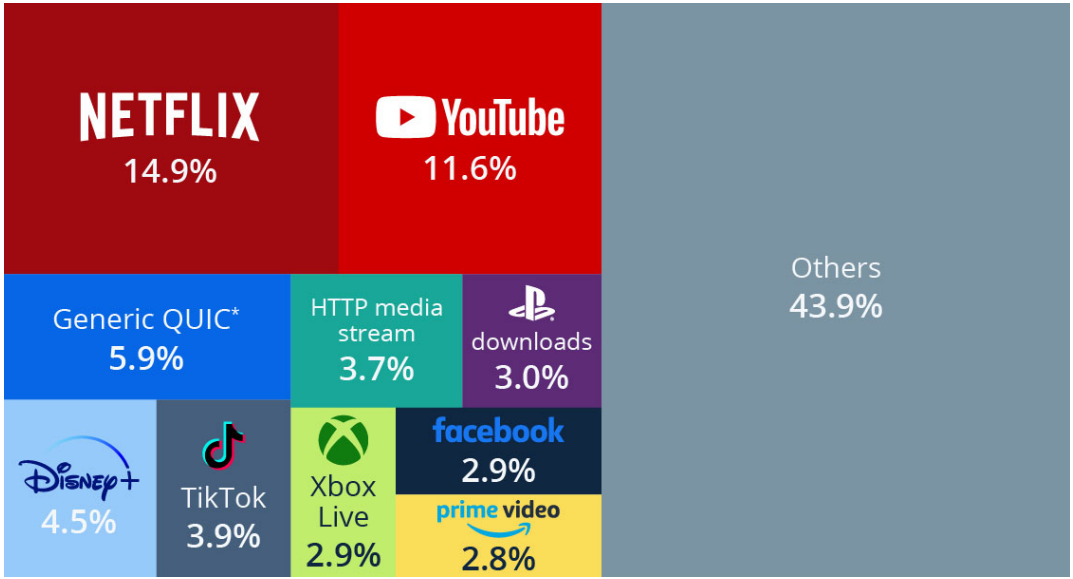
Video streaming applications and content delivery networks

Share of traffic per application category for (unnamed) European ISP



Source: Ericsson Mobility Report, https://www.ericsson.com/en/reports-and-papers/mobility-report/articles/short-form-dominates-video-traffic

Percentage of global Internet traffic per application category in 2022



Source: Statista, "Netflix is Responsible for 15% of Global Internet Traffic," March 2023.

Video characteristics

- A video is a sequence of (encoded/compressed) images
  - Images are displayed at a certain rate, expressed as frames per second (FPS), such as 24, 30, or 60
  - The size of the images (in bytes) determines the bitrate of the video, expressed as (Megabits) bits per second ((M)bps)
- The same video can be encoded in multiple qualities/bitrates (this is the basis of adaptive streaming)

24 FPS, 1080p (1920x1080 pixel), 5 Mbps

24 FPS, 4K (3840x2160 pixel), 20 Mbps

Given a movie with an average bitrate of 5 Mbps, and a duration of 90 minutes, what is the approximate total size of the video?

3.3 Megabyte 0

27 Megabyte 0

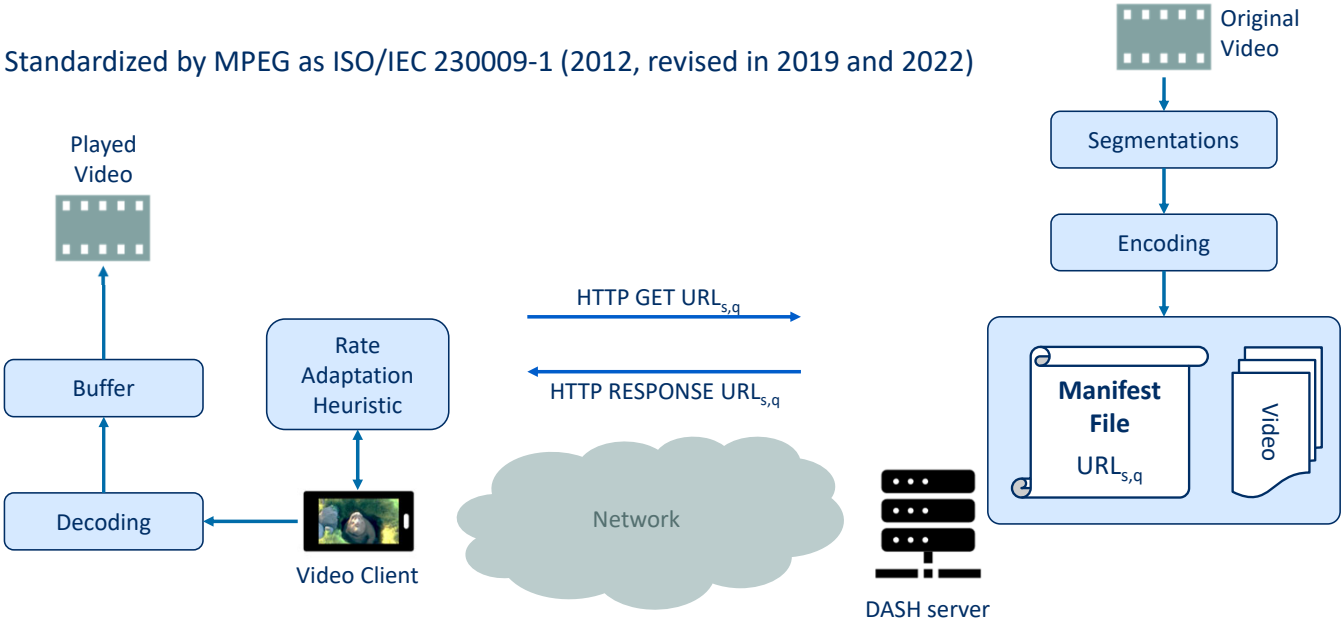
3.3 Gigabyte 0

27 Gigabyte 0

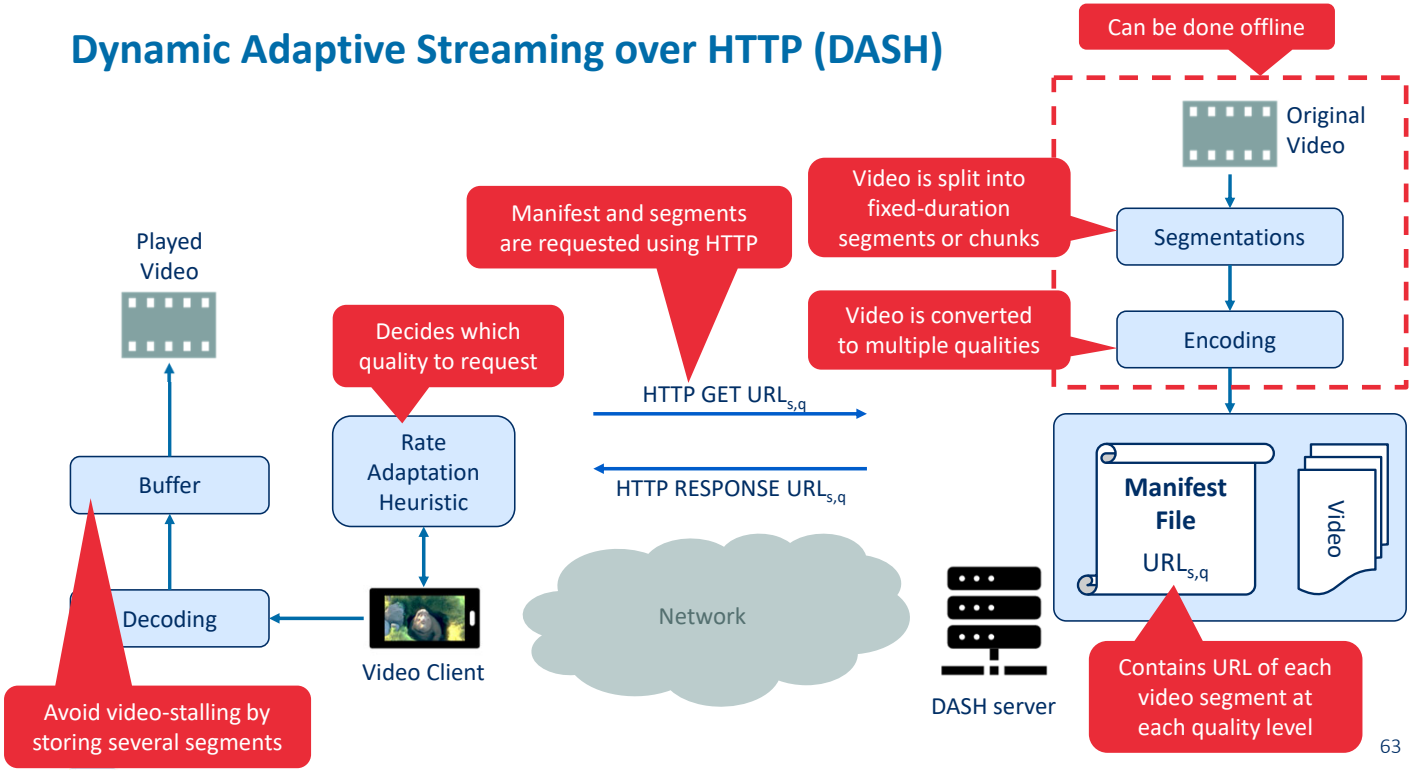
None of the above 0

Dynamic Adaptive Streaming over HTTP (DASH)

Standardized by MPEG as ISO/IEC 230009-1 (2012, revised in 2019 and 2022)



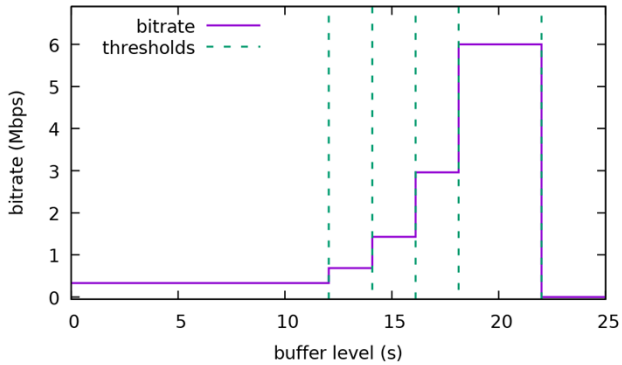
Dynamic Adaptive Streaming over HTTP (DASH)



Rate Adaptation Heuristic

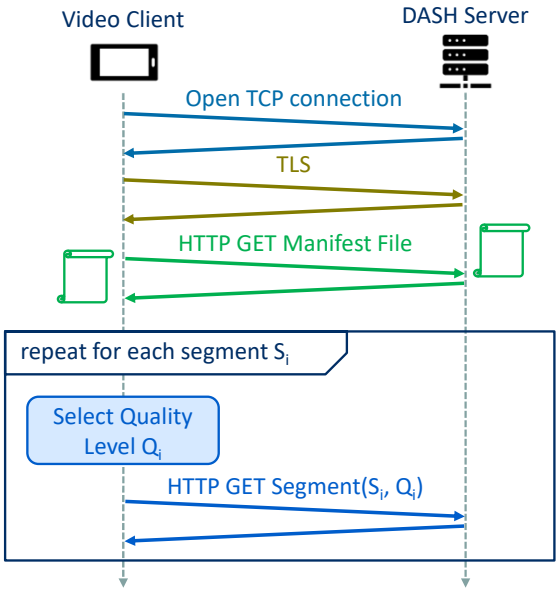
- Rate adaptation heuristics are “vendor specific” (i.e., they are not standardized)
- Goal: Find a balance between two important Quality of Experience metrics:
  - The **video quality** should be maximized (and (optionally) fluctuations in quality minimized)
  - The **rebuffering time** (i.e., waiting time due to an empty playout buffer) should be minimized
- Information that can be used by the client:
  - Buffer level (i.e., playout duration of buffered segments)
  - Estimation of historical bandwidth (i.e., by dividing segment size by segment download time)
  - Previously requested quality levels

Example of selected video bitrate based on buffer level thresholds ((Buffer Occupancy based Lyapunov Algorithm):



Source: *K. Spiteri, R. Urgaonkar and R. K. Sitaraman, "BOLA: Near-Optimal Bitrate Adaptation for Online Videos," in IEEE/ACM Transactions on Networking, vol. 28, no. 4, pp. 1698-1711, Aug. 2020, doi: 10.1109/TNET.2020.2996964.*

Typical HTTP message exchange for DASH



Example DASH manifest file (MD):

```
<?xml version="1.0" encoding="UTF-8"?>
<MPD xmlns="urn:mpeg:dash:schema:mpd:2011" minBufferTime="PT1.500000S" type="static"
mediaPresentationDuration="PT0H9M56.46S" profiles="urn:mpeg:dash:profile:isoff-
live:2011">
 <ProgramInformation moreInformationURL="http://gpac.sourceforge.net">
  <Title>dashed/BigBuckBunny_10s_simple_2014_05_09.mpd generated by GPAC</Title>
 </ProgramInformation>
 <Period duration="PT0H9M56.46S">
  <AdaptationSet segmentAlignment="true" group="1" maxWidth="480" maxHeight="360"
      maxFrameRate="24" par="4:3">
   <SegmentTemplate timescale="96" media="$Bandwidth$bps/BigBuckBunny_10s$Number$.m4s"
      startNumber="1" duration="960"
      initialization="bunny_$Bandwidth$bps/BigBuckBunny_10s_init.mp4" />
   <Representation id="320x240 45.0kbps" mimeType="video/mp4" codecs="avc1.42c00d"
      width="320" height="240" frameRate="24" startWithSAP="1" bandwidth="45373" />
   <Representation id="1280x720 987.0kbps" mimeType="video/mp4" codecs="avc1.42c01f"
      width="1280" height="720" frameRate="24" startWithSAP="1" bandwidth="987061" />
   <Representation id="1920x1080 3.8Mbps" mimeType="video/mp4" codecs="avc1.42c032"
      width="1920" height="1080" frameRate="24" startWithSAP="1" bandwidth="3792491" />
  </AdaptationSet>
 </Period>
</MPD>
```

Scalable video streaming using Content Delivery Networks (CDNs)

Naïve approach: A single centralized datacenter

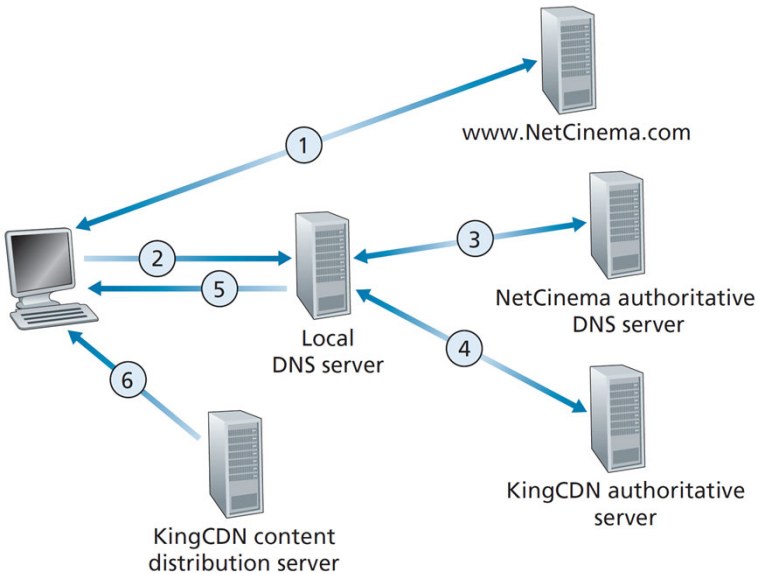
Disadvantages:

- High delay (packets cross many links and ISPs)
- Popular videos are sent many times over the same links
- Single point-of-failure

Scalable approach: Distributed CDN

- Types of CDNs
  - **Private CDN**: Owned by a content provider (e.g., Netflix, Google)
  - **Third-party CDN**: Distributes on behalf of a provider (e.g., Akamai)
- Server placement philosophies
  - **Enter Deep:** Deploy servers in ISP access networks (near end users)
  - **Bring Home**: Build larger clusters at Internet Exchange Points (IXPs)

CDN operation based on DNS interception and redirection



1. User visits NetCinema webpage
2. DNS request for selected video hostname is sent to local DNS server
3. NetCinema DNS server detects it's a video URL and forwards to KingCDN
4. KingCDN private DNS infrastructure forwards user to its own content distribution server
5. Local DNS server forwards IP address of KingCDN content server to user host
6. User downloads content from KingCDN content server via HTTP

Cluster selection strategy

- Based on the IP address of the client's local DNS server
- Strategy is usually *proprietary*
- Common strategies:

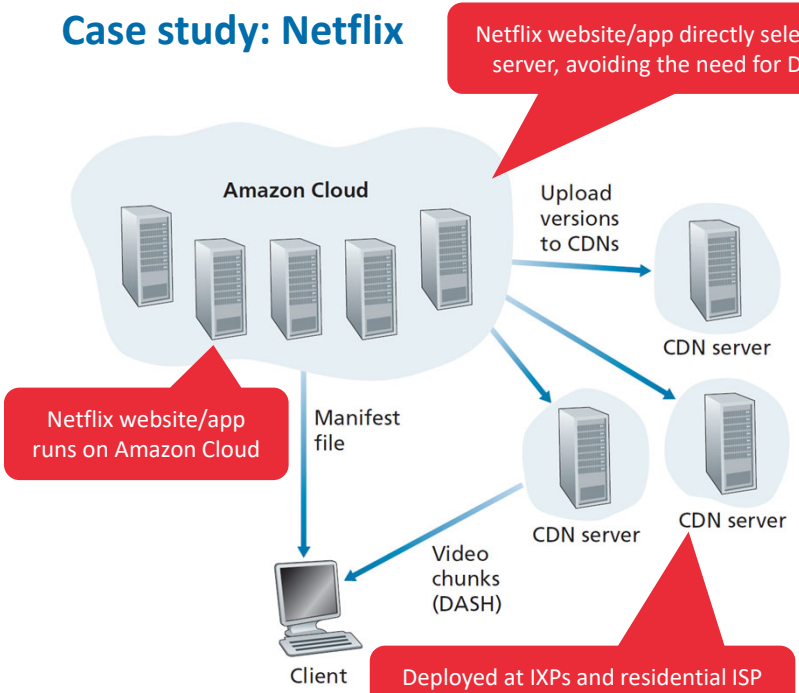
Geographical closeness

- Select content server geographically nearest to the local DNS server
- Geo-location databases map IP addresses to geographical locations
- Disadvantages
  - Geographic closeness may not reflect network closeness
  - Some users use remotely located local DNS servers
  - Ignores variations in delay and bandwidth over time

Real-time measurements

- CDN clusters periodically measure delay and loss to local DNS servers using probes (e.g., ping messages)
- Disadvantages
  - Local DNS servers may not be configured to respond to probes
  - Significant overhead to frequently ping all possible local DNS servers

Case study: Netflix



Amazon Cloud functions

- Netflix website (user management, billing, movie catalogue)
- Content ingestion (studio master versions)
- Content processing (transform master into diverse bit rates for different devices)
- Uploading content versions to private CDN

Netflix private CDN

- Deployed at > 200 IXP locations (contain full Netflix catalogue)
- Residential ISPs can deploy a (free) Netflix server rack (contain popular content)
- Content is pushed during off-peak hours

Summary

Summary

- The **application layer** consists of network applications and application-layer protocols
  - The run only on end hosts (and not on routers or switches)
  - Sockets provide an API to transmit or receive data via the transport layer (e.g., UDP or TCP)
- The **HyperText Transfer Protocol (HTTP)** enables communication between web clients and servers
  - It makes use of TCP to ensure reliable end-to-end communication
  - HTTP requests are used to request web pages and objects, responses transfer objects back to the client
  - Persistent connections and pipelining reduce latency
  - HTTP/2 offers additional features (multiplexing, prioritization, push and compression) to further reduce perceived latency
- The **Simple Mail Transfer Protocol (SMTP)** enables communication between mail servers
  - It is a push-based protocol for transmitting email messages towards the recipients mail server
  - Other protocols are needed to retrieve email from the mail server's mailbox (e.g., HTTP or IMAP)
- The **Domain Name System (DNS)** maps human-readable hostnames to router-readable IP addresses
  - It is based on a hierarchy of DNS servers
  - It also offers aliasing and load distribution functionality
- **Video streaming applications** use HTTP in combination with Content Delivery Networks for scalable content delivery
- The contents of this part is covered in the **book** in Sections 2.1, 2.2, 2.3, 2.4, and 2.6

University of Antwerp
Faculty of Science

End of Part 2